Containerized ETL Pipeline for Restaurant Trend Analysis

Şeyma Betül İSKENDER
Department of Artificial Intelligence and Data Engineering
Istanbul Technical University
Istanbul, Turkey
iskenders22@itu.edu.tr

Ahmet Selim ÖZEL
Department of Artificial Intelligence and Data Engineering
Istanbul Technical University
Istanbul, Turkey
ozelah23@itu.edu.tr

***Abstract*—This report presents a fully containerized data engineering pipeline for restaurant trend analysis. The system combines a public Zomato restaurant dataset with synthetic timestamped order events generated by a Python service. Cleaned restaurant data and generated order events are stored in PostgreSQL, while Apache Airflow orchestrates record verification, aggregation, and indexing tasks. Aggregated city- and cuisine-level summaries are indexed into Elasticsearch and visualized through Kibana dashboards. The entire system runs locally using Docker Compose, with team-authored Python code executed inside containers. The project demonstrates an end-to-end workflow from raw CSV data to interactive analytical dashboards while addressing reproducibility, persistent storage, workflow orchestration, retry logic, and basic failure handling.**

***Index Terms*—ETL pipeline, Docker, PostgreSQL, Apache Airflow, Elasticsearch, Kibana, synthetic data, restaurant analytics**

I. Executive Summary

This project implements an end-to-end containerized data engineering pipeline for restaurant trend analysis. The pipeline ingests restaurant metadata from a public Zomato dataset, enriches the workflow with synthetic timestamped order events, stores structured data in PostgreSQL, orchestrates analytical processing with Apache Airflow, indexes aggregated summaries into Elasticsearch, and visualizes the results in Kibana.

The final system runs through Docker Compose and includes PostgreSQL, pgAdmin, Apache Airflow, Elasticsearch, Kibana, and a team-authored Python application container. The implemented workflow produces 50,000 synthetic order events linked to real restaurant records and generates city-cuisine analytical summaries. These summaries are then indexed as Elasticsearch documents and used to build Kibana dashboard panels such as order volume by city, cuisine order share, restaurant density by city, and average order value by city.

The main goal of the project is not to process massive data, but to demonstrate a reproducible and defensible data engineering architecture that connects ingestion, storage, orchestration, indexing, and visualization layers in a coherent local environment.

II. Problem Statement and Motivation

Restaurant datasets often contain heterogeneous information such as restaurant names, cities, cuisines, ratings, delivery availability, price indicators, and location-related attributes. While these records are useful for descriptive analysis, a static dataset alone does not demonstrate a full data engineering workflow. Therefore, this project extends the restaurant metadata with synthetic timestamped order events in order to simulate activity over time. The problem addressed by this project is the design and implementation of a reproducible data pipeline that transforms raw restaurant data into analytical outputs.

This project is motivated by common data engineering scenarios in which raw business data is cleaned, enriched with event data, transformed into analytical summaries, and delivered to a dashboarding layer.

III. Architectural Design

A. Overall Architecture

The system is organized into five main layers: data sources, relational storage, workflow orchestration, search/indexing, and visualization. Fig. 1 shows the high-level architecture.

Fig. 1: Containerized restaurant analytics pipeline architecture.

The pipeline starts with CSV-based restaurant data and a synthetic order generator. Restaurant and order records are stored in PostgreSQL. Apache Airflow executes a DAG that verifies record counts, builds analytical summaries, and indexes the resulting summary data into Elasticsearch. Kibana consumes the Elasticsearch index and provides visual dashboard panels.

B. Data Flow

The implemented data flow is as follows:

1) The Python loading script reads `zomato.csv` and `Country-Code.csv`.

2) Cleaned restaurant records are inserted into the PostgreSQL `restaurants` table.
3) A Python synthetic order generator creates 50,000 timestamped order events and inserts them into the `orders` table.
4) Apache Airflow checks whether restaurant and order data exist.
5) Airflow builds the `city_cuisine_summary` table by joining restaurants and orders.
6) Airflow runs the indexing script, which writes summary rows into the Elasticsearch `city_cuisine_summary` index.
7) Kibana visualizes the indexed summary documents.

This structure separates raw entity data, event data, analytical summaries, and visualization-ready documents.

C. Rationale for Design Decisions

PostgreSQL was selected as the relational storage layer because restaurant records and order events have clear structured schemas and relational links through `restaurant_id`. Apache Airflow was selected to make the workflow explicit, repeatable, and monitorable. Elasticsearch was used as the indexing and analytics layer for fast filtering and dashboard queries. Kibana was chosen as the visualization interface because it integrates directly with Elasticsearch. Docker Compose was used to make the multi-service system reproducible in a local environment.

The architecture intentionally avoids unnecessary production complexity such as Kafka, Kubernetes, CeleryExecutor, or distributed processing frameworks. The objective is a reliable student-scale project that can be run, demonstrated, and defended during evaluation.

IV. Tools Used and Rationale

Table I summarizes the main tools used in the project.

TABLE I: Tools used in the project

Tool	Role in the Project
Docker Compose	Defines and runs all services in a local containerized environment.
PostgreSQL	Stores restaurant records, synthetic order events, and analytical summaries.
pgAdmin	Provides a web interface for database inspection and demo queries.
Apache Airflow	Orchestrates record verification, aggregation, retry handling, and indexing tasks.
Elasticsearch	Stores summary documents for search and dashboard analytics.
Kibana	Provides visual dashboards over Elasticsearch-indexed summaries.
Python	Implements loading, synthetic event generation, and indexing logic.
Faker	Supports synthetic timestamped order generation.

Compared with a notebook-based analysis, this architecture provides better reproducibility and clearer separation of responsibilities. Compared with a more complex streaming architecture, it is easier to run locally and better suited to the project constraints.

V. Implementation Details

A. Repository Structure

The project repository is organized as follows:

```
.
dags/
    restaurant_pipeline.py
data/
    zomato.csv
    Country-Code.csv
docs/
    pipeline-diagram.png
postgres/
    init.sql
scripts/
    load_restaurants.py
    generate_orders.py
    index_summary.py
.env.example
.gitignore
docker-compose.yml
Dockerfile
requirements.txt
README.md
```

B. PostgreSQL Schema

Fig. 2 shows the database schema including all application-level tables and their relationships.

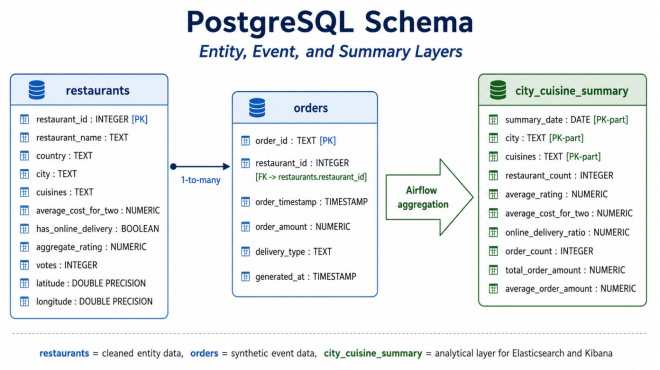

Fig. 2: PostgreSQL schema showing restaurants, orders, and city_cuisine_summary tables.

The PostgreSQL database contains four application-level tables:

- `restaurants`: stores cleaned restaurant records from the Zomato dataset;
- `invalid_restaurants`: stores rejected records for later inspection;
- `orders`: stores synthetic timestamped order events;
- `city_cuisine_summary`: stores aggregated analytical metrics.

The `restaurants` table includes fields such as restaurant name, country, city, cuisine, average cost, online delivery

availability, rating, vote count, latitude, and longitude. The `orders` table contains synthetic order events linked to restaurants through `restaurant_id`. The summary table groups records by date, city, and cuisine and stores metrics such as restaurant count, average rating, order count, total order amount, and average order amount.

C. Synthetic Order Generation

The synthetic order generator creates 50,000 order events. Each order is linked to an existing restaurant record. The generator uses weighted restaurant selection so that restaurants with higher ratings, more votes, and online delivery availability receive more synthetic orders. Order timestamps are generated over a recent time window with higher probability around lunch and dinner hours. Order amounts are derived from the restaurant's average cost field to create more realistic spending patterns.

The generation step is idempotent. If the `orders` table already contains the configured number of records, the script skips generation rather than duplicating rows. This prevents repeated `docker compose up --build` executions from inflating the dataset.

D. Airflow DAG Structure

The Airflow DAG is named `restaurant_pipeline`. It contains four sequential tasks:

1) `check_restaurants`: verifies that restaurant records exist;
2) `check_orders`: verifies that the order event threshold is satisfied;
3) `build_summary`: computes city-cuisine summary metrics;
4) `index_summary`: indexes summary documents into Elasticsearch.

The DAG uses retry logic with three retries (`retries=2`) and a one-minute retry delay (`retry_delay=timedelta(minutes=1)`). This was added to address temporary failures such as delayed service availability or transient indexing errors. During development, a NULL cuisine value caused the summary task to fail. The issue was resolved with `COALESCE(cuisines, 'Unknown')`, and the retry mechanism helped expose the failure handling behavior.

E. Elasticsearch Index Design

The Elasticsearch index is named `city_cuisine_summary`. Each PostgreSQL summary row is converted into a JSON document. The document contains fields such as:

- `summary_date`;
- `city`;
- `cuisines`;
- `restaurant_count`;
- `average_rating`;
- `average_cost_for_two`;
- `online_delivery_ratio`;
- `order_count`;
- `total_order_amount`;
- `average_order_amount`.

A stable document ID is derived from city and cuisine values. This makes the indexing step repeatable because rerunning the DAG updates existing documents rather than creating uncontrolled duplicates.

F. Kibana Dashboard

Kibana is used to visualize the Elasticsearch index. The dashboard includes four panels built on the `city_cuisine_summary` index. Fig. 3 through Fig. 6 show each panel individually.

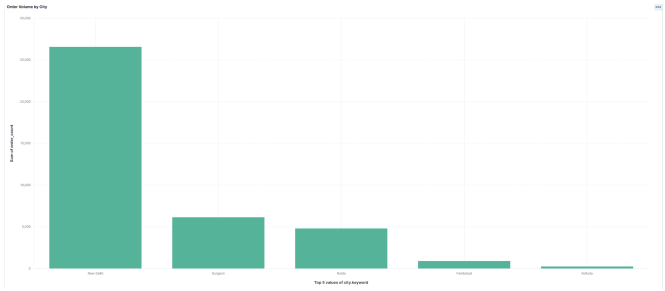

Fig. 3: Order Volume by City. New Delhi dominates with over 26,000 synthetic orders, far ahead of Gurgaon and Noida, reflecting both the high restaurant density and the weighted order generation logic.

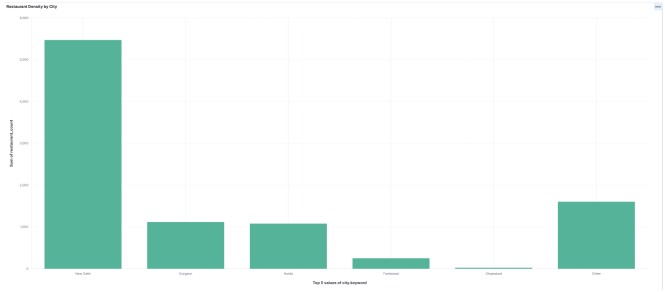

Fig. 4: Restaurant Density by City. New Delhi has approximately 5,400 restaurants, accounting for the majority of indexed records. This distribution directly influences order volume patterns shown in Fig. 3.

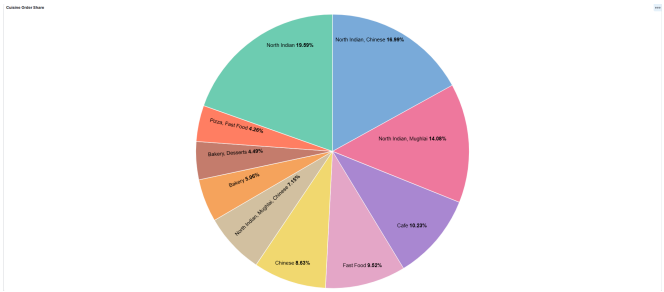

Fig. 5: Cuisine Order Share. North Indian cuisine leads with 19.59% of total orders, followed by North Indian-Chinese at 16.99% and North Indian-Mughlai at 14.08%. Indian cuisine variants collectively account for over 50% of all orders.

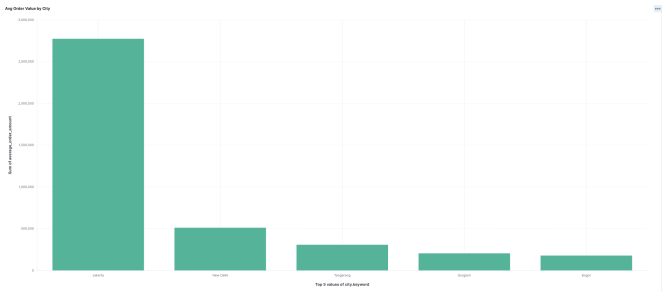

Fig. 6: Average Order Value by City. Jakarta shows a significantly higher average order value compared to Indian cities. This anomaly is expected given that order amounts are derived from the restaurant's average cost field, and Jakarta records use a different currency scale than Indian records.

VI. Evaluation

A. Functional Validation

The system was validated by checking each major stage of the pipeline:

- PostgreSQL contains 9,551 valid restaurant records;
- PostgreSQL contains 50,000 synthetic order events;
- Airflow DAG tasks complete successfully;
- `city_cuisine_summary` contains 3,031 summary rows;
- Elasticsearch contains 3,031 indexed summary documents;
- Kibana reads the Elasticsearch index and displays dashboard panels.

The pipeline successfully transformed 9,551 raw restaurant records and 50,000 synthetic order events into 3,031 city-cuisine summary documents, demonstrating end-to-end data flow from ingestion to visualization. Each pipeline stage produced the expected outputs, and no data was lost between layers. The Airflow DAG completed all four tasks without manual intervention, and Kibana dashboard panels loaded correctly from the Elasticsearch index.

B. Data Quality Checks

The loading process validates coordinate fields and stores invalid records separately when needed. Missing cuisine values were identified during aggregation and handled with a fallback value of `Unknown`. This prevents NULL values from violating summary table constraints.

C. Idempotency

The project includes idempotency mechanisms at multiple layers. Restaurant records use deterministic restaurant IDs. Synthetic order generation skips execution when the expected number of orders already exists. The Elasticsearch indexing script uses stable document IDs derived from city and cuisine values, allowing repeated indexing runs to update existing documents.

D. Failure Handling

Airflow tasks are configured with retry logic (`retries=2`, `retry_delay=timedelta(minutes=1)`). This allows temporary failures to be retried automatically. The most visible development-time failure occurred in the summary aggregation step due to NULL cuisine values. The failure was diagnosed through Airflow logs and fixed with SQL-level NULL handling using `COALESCE(cuisines, 'Unknown')`. This demonstrates the value of task-level observability and retry behavior in an orchestrated workflow.

VII. Limitations and Future Work

The current implementation provides a reliable end-to-end local pipeline, but it has several limitations.

First, the restaurant dataset is static, and order events are synthetic. Although the synthetic orders are linked to real restaurant records and generated with weighted logic, they do not represent real customer behavior. Second, the project uses city- and cuisine-level analytical summaries rather than true real-time streaming. Third, the dashboard provides location-aware city-level analysis but does not implement a full map-based geospatial visualization. Fourth, Elasticsearch is used for summary indexing and dashboarding, but advanced search features and geo-point mappings are left as future work.

Future improvements may include adding an Apache NiFi ingestion flow, indexing restaurant-level geo-point documents, building a Kibana Maps dashboard, adding automated tests, and exporting Kibana saved objects for easier dashboard reproducibility.

VIII. AI Usage Declaration

AI tools were used as supplementary assistance during the project. They were used for brainstorming architectural alternatives, debugging Docker and Airflow issues, improving report language, and discussing how to simplify the implementation into a reliable student-scale system. AI suggestions were not copied without validation. All code and configuration changes were manually reviewed, adapted, executed locally, and validated through PostgreSQL queries, Airflow DAG runs, Elasticsearch search results, and Kibana dashboards.

The final project, implementation decisions, debugging, and demo preparation were completed and verified by the team.

IX. Individual Contribution Table

TABLE II: Individual contribution summary

Team Member	Contributions
Şeyma Betül İSKENDER	Worked on project planning, report preparation, dashboard interpretation, Kibana dashboard design, documentation, and validation of analytical outputs.
Ahmet Selim ÖZEL	Worked on Docker Compose setup, PostgreSQL schema, Python ETL scripts, Airflow DAG, Elasticsearch indexing, and debugging.

X. References

References

[1] S. Mehta, "Zomato Restaurants Data," Kaggle, 2019. [Online]. Available: https://www.kaggle.com/datasets/shrutimehta/zomato-restaurants-data

[2] The PostgreSQL Global Development Group, "PostgreSQL Documentation." [Online]. Available: https://www.postgresql.org/docs/

[3] Apache Software Foundation, "Apache Airflow Documentation." [Online]. Available: https://airflow.apache.org/docs/

[4] Elastic, "Elasticsearch Guide." [Online]. Available: https://www.elastic.co/guide/en/elasticsearch/reference/current/index.html

[5] Elastic, "Kibana Guide." [Online]. Available: https://www.elastic.co/guide/en/kibana/current/index.html

[6] Joke2k, "Faker: Python package that generates fake data." [Online]. Available: https://faker.readthedocs.io/